University of Liberal Arts Bangladesh

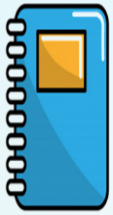
Department of Computer Science & Engineering

CSE 3203 - Software Engineering

Lecture Unlucky 13:
Software Design Patterns Part 1

Atanu Shuvam Roy

Lecturer,
Dept. of CSE, ULAB, Dhaka



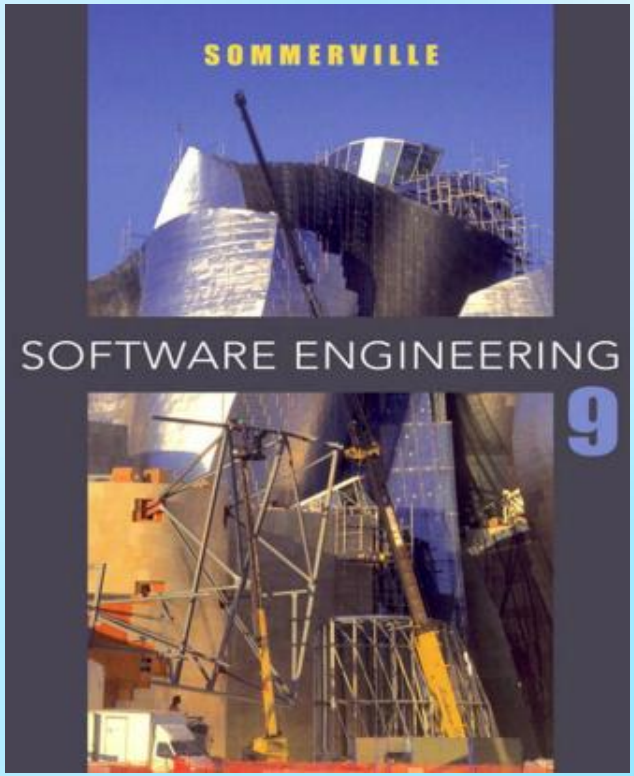
EMAIL:
atanu.shuvam@ulab.edu.bd

Class Code
byvb74r2

Fall 2025

Book for Reference

- Software Engineering 9th Edition by Ian Sommerville



Mark Distribution

- Mid Semester Exam – 25%
- Final Semester Exam – 40%
- Class Attendance & Performance – 10%
- Course Project – 25%

Is it possible to get A/A+ ?

- Absolutely. Just follow my instructions to the point.

How to fail in this course ?

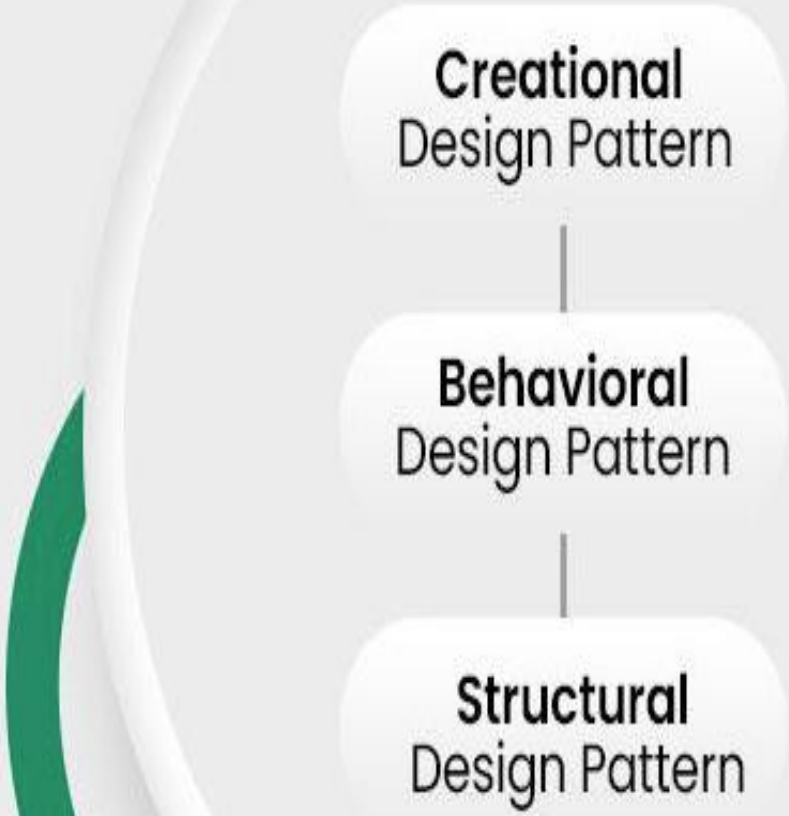
- Don’t follow my instructions.

Instructions

Types of

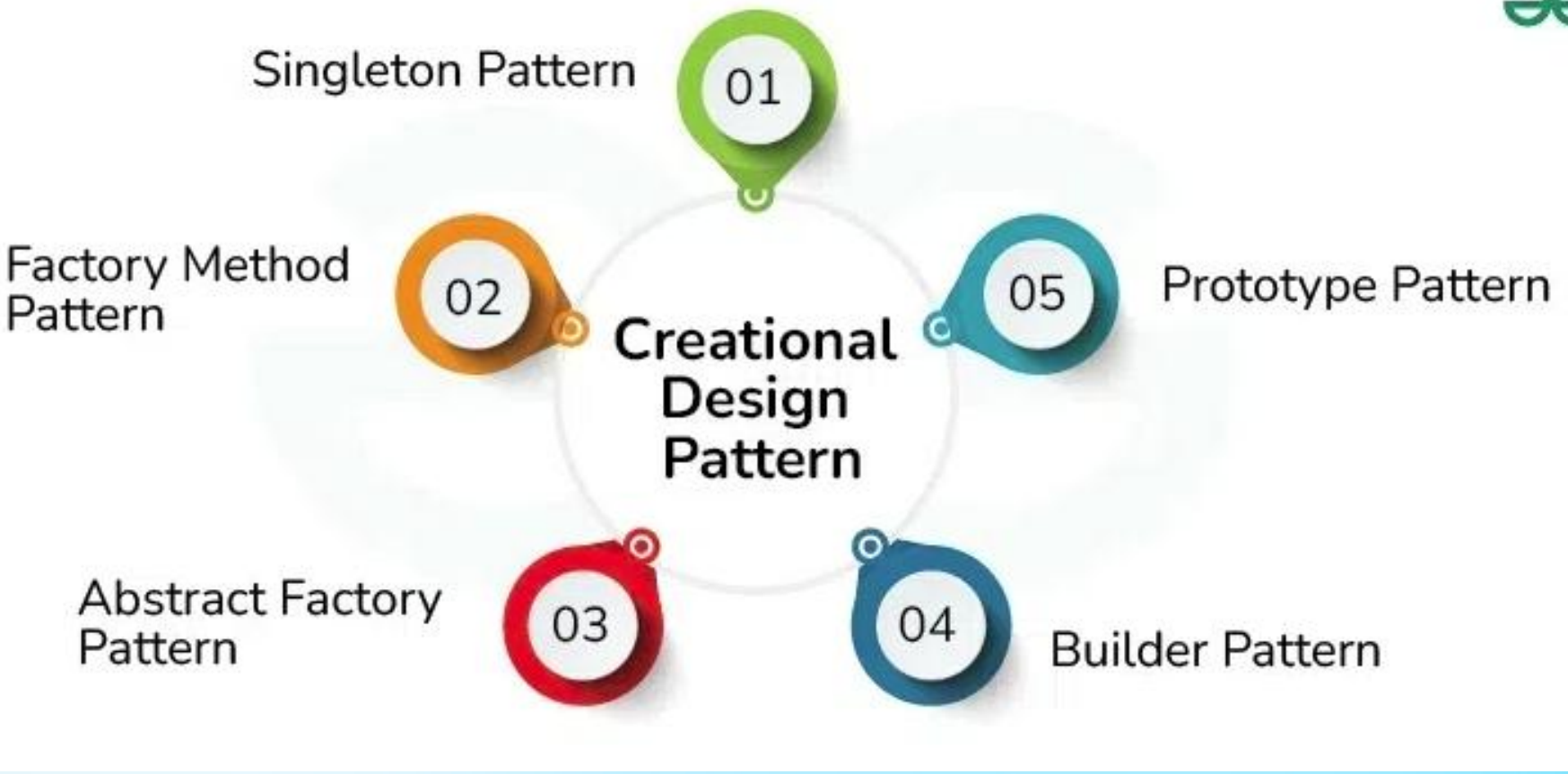
Software

Design Pattern



Creational

Design Pattern


Singleton Pattern
01
Factory Method Pattern
02
Creational Design Pattern
05
Prototype Pattern
Abstract Factory Pattern
03
04
Builder Pattern

Creational Design Pattern

- **1.** Factory Method Design Pattern
  - This pattern is typically helpful when it's necessary to separate the construction of an object from its implementation. With the use of this design pattern, objects can be produced without having to define the exact class of object to be created. Below is when to use Factory Method Design Pattern:
  - A class can't anticipate the class of objects it must create.
  - A class wants its subclass to specify the objects it creates.
  - Classes delegate responsibility to one of several helper subclasses, and you want to localize the knowledge of which helper subclass is the delegate.

Creational Design Pattern

2. The Builder Design Pattern:

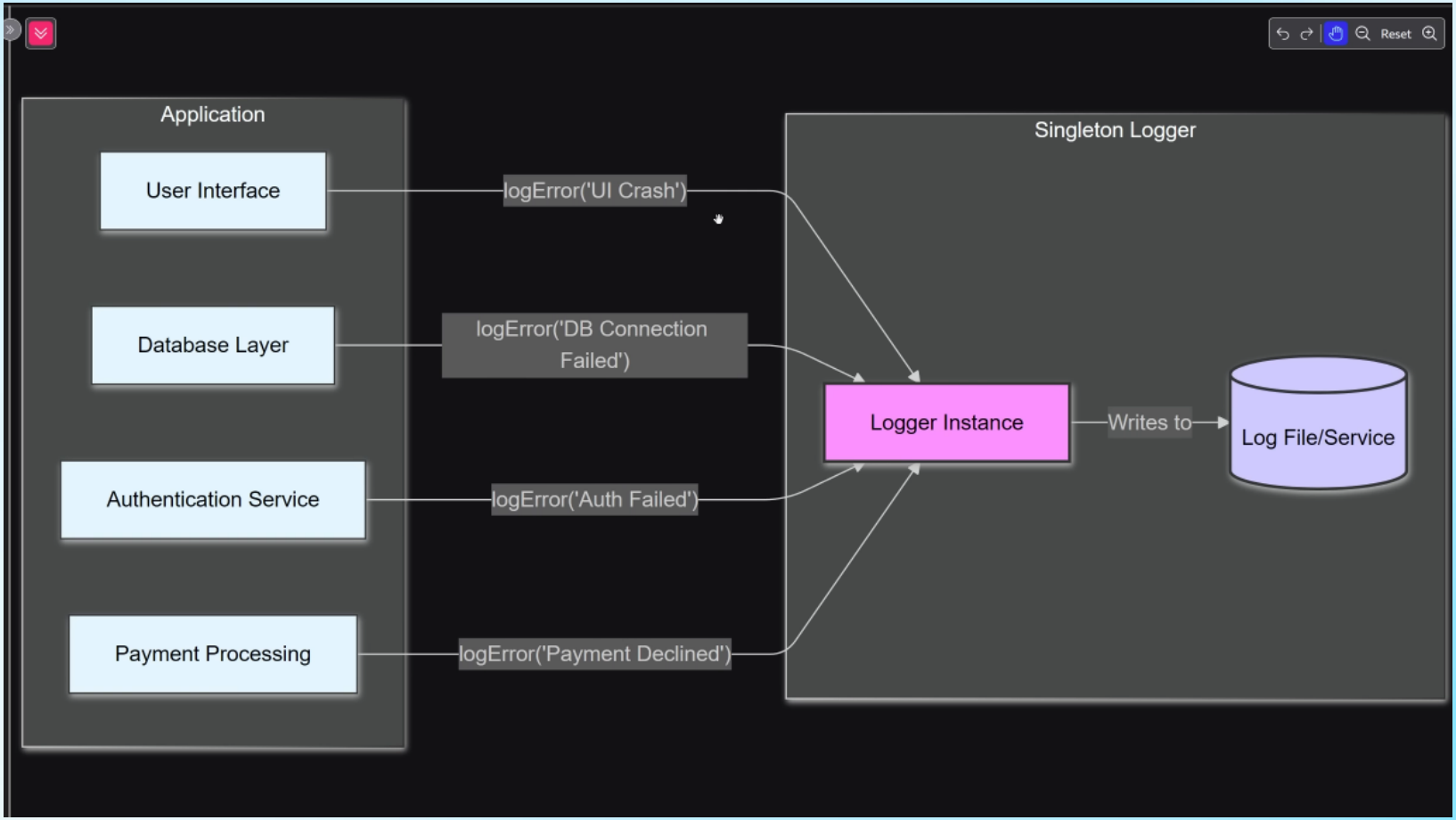
The Builder Design Pattern is a creational design pattern that provides a step-by-step approach to constructing complex objects. It separates the construction process from the object's representation, enabling the same method to create different variations of an object.

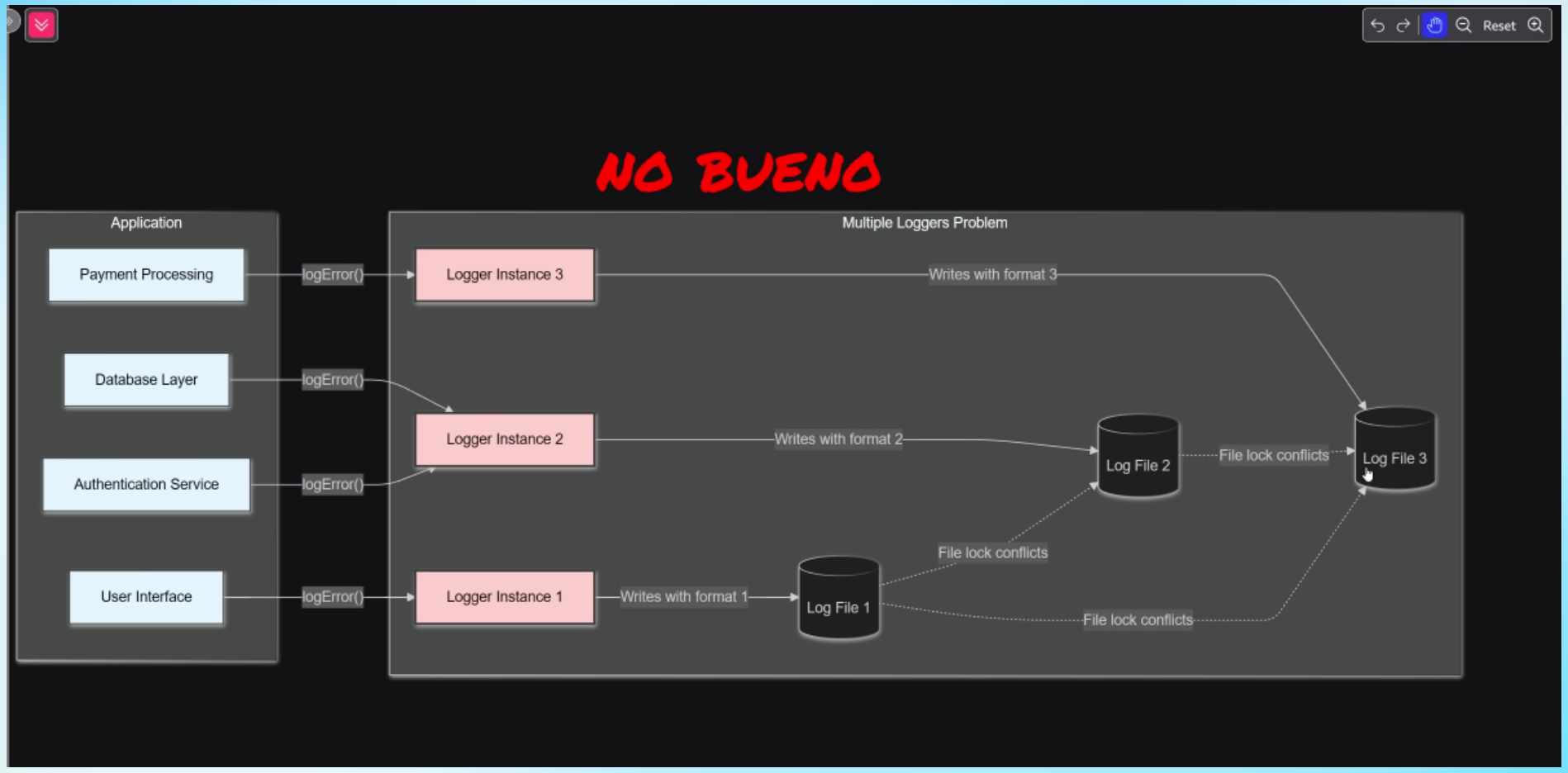
- Encapsulates object construction logic in a separate Builder class.
- Allows flexible and controlled object creation.
- Supports different variations of a product using the same process.
- Improves readability and maintainability by avoiding long constructors with many parameters.

Creational Design Pattern

- **3.** Singleton Method Design Pattern
  - The Singleton method or Singleton Design pattern is one of the simplest design patterns. It ensures a class only has one instance, and provides a global point of access to it. Below is when to use Singleton Method:
  - There must be exactly one instance of a class, and it must be accessible to clients from a well-known access point.
  - When the sole instance should be extensible by subclassing, and clients should be able to use an extended instance without modifying their code.

Singleton Design Pattern




NO BUENO
Application
Payment Processing
Database Layer
Authentication Service
User Interface
logError()
logError()
logError()
logError()
Multiple Loggers Problem
Logger Instance 3
Logger Instance 2
Logger Instance 1
Writes with format 3
Writes with format 2
Writes with format 1
Log File 1
Log File 2
Log File 3
File lock conflicts
File lock conflicts
File lock conflicts
Reset

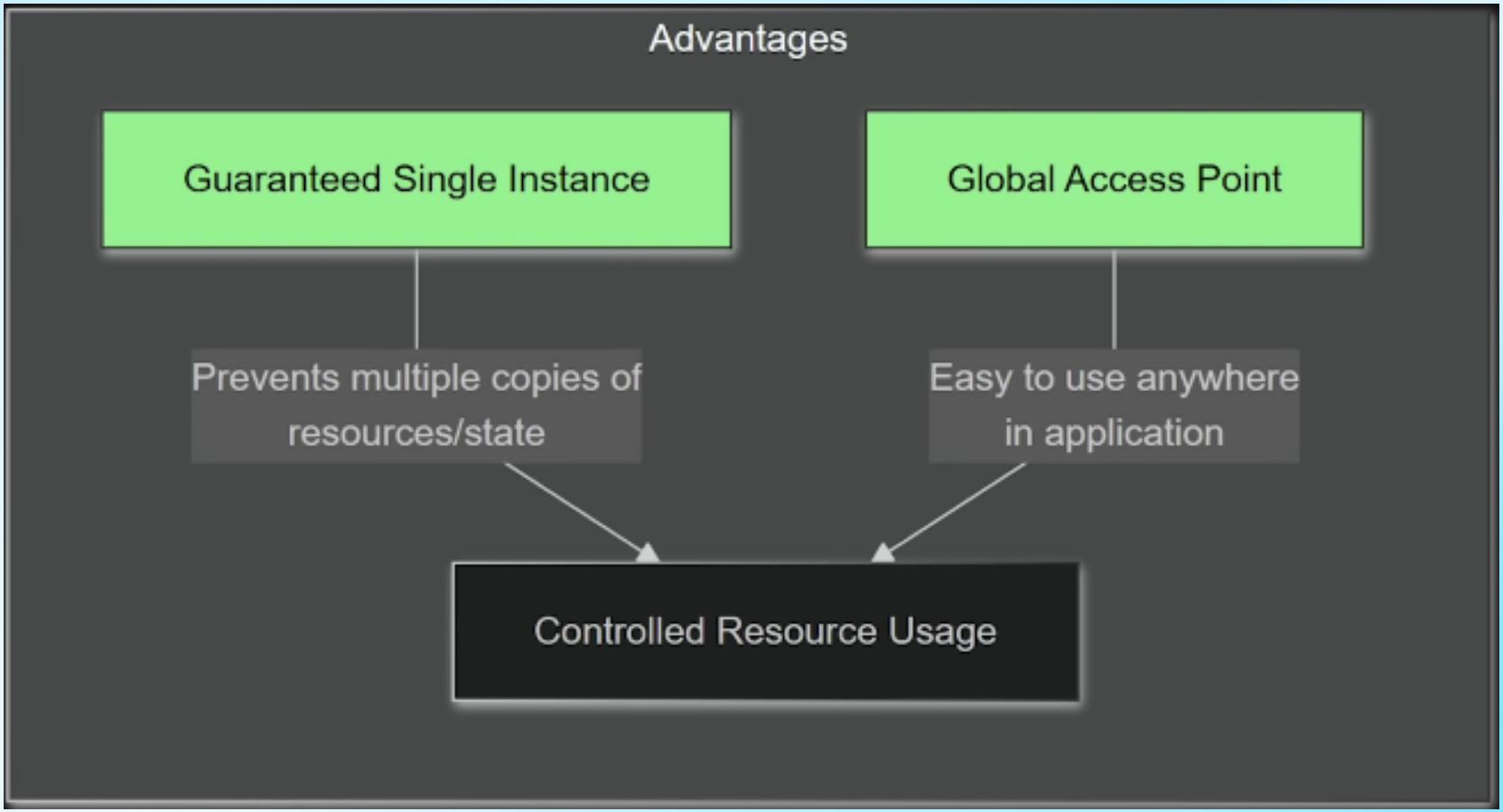
Bad vs Good (singleton)

```
1  // Bad: Multiple loggers creating chaos
2  const logger1 = new Logger();
3  const logger2 = new Logger(); // Another logger writing
   to same file!?
4
5  // Good: Single logger everyone uses
6  const logger = Logger.getInstance();
7  logger.error('Failed to process payment');
```

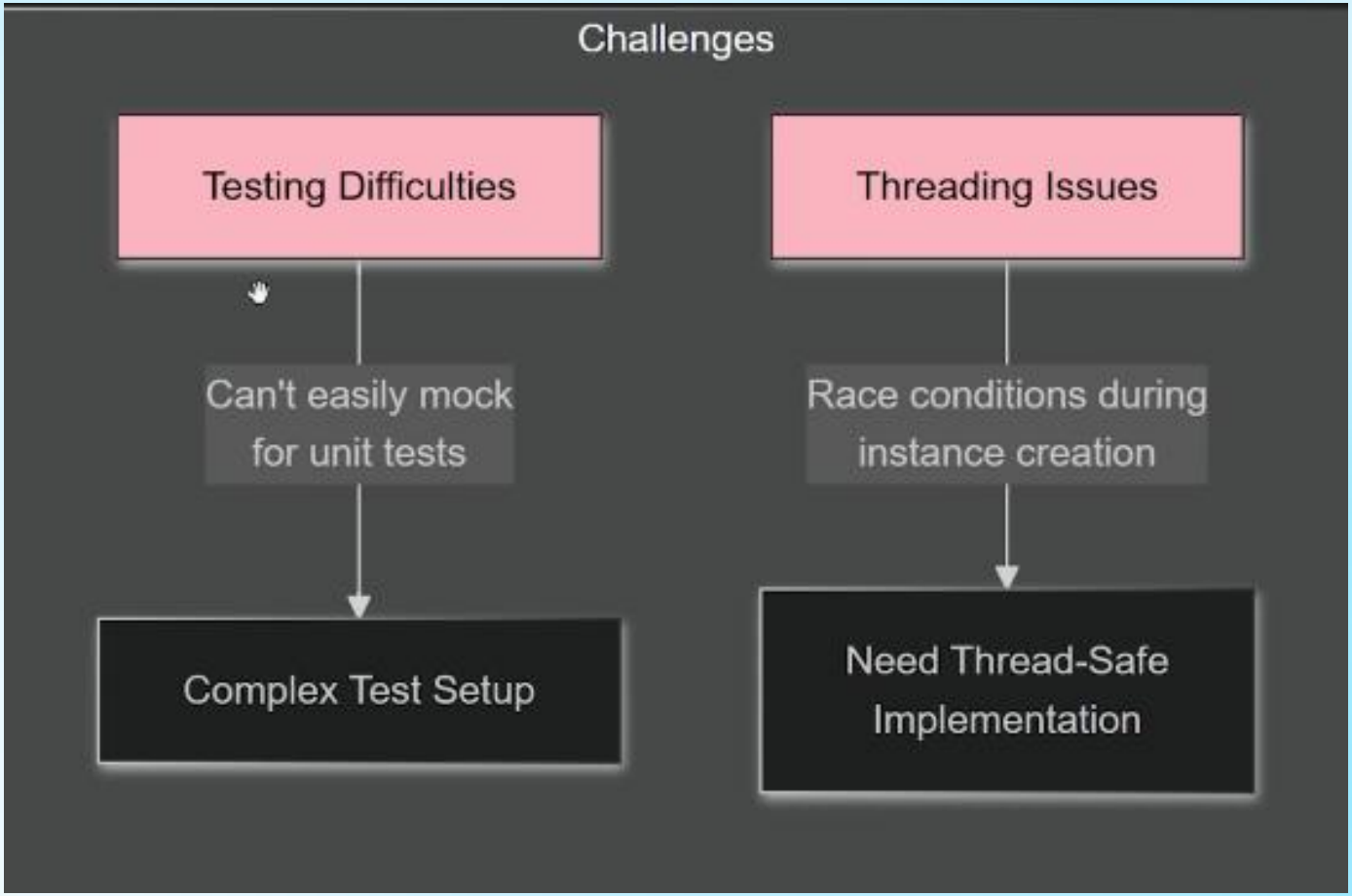
Example

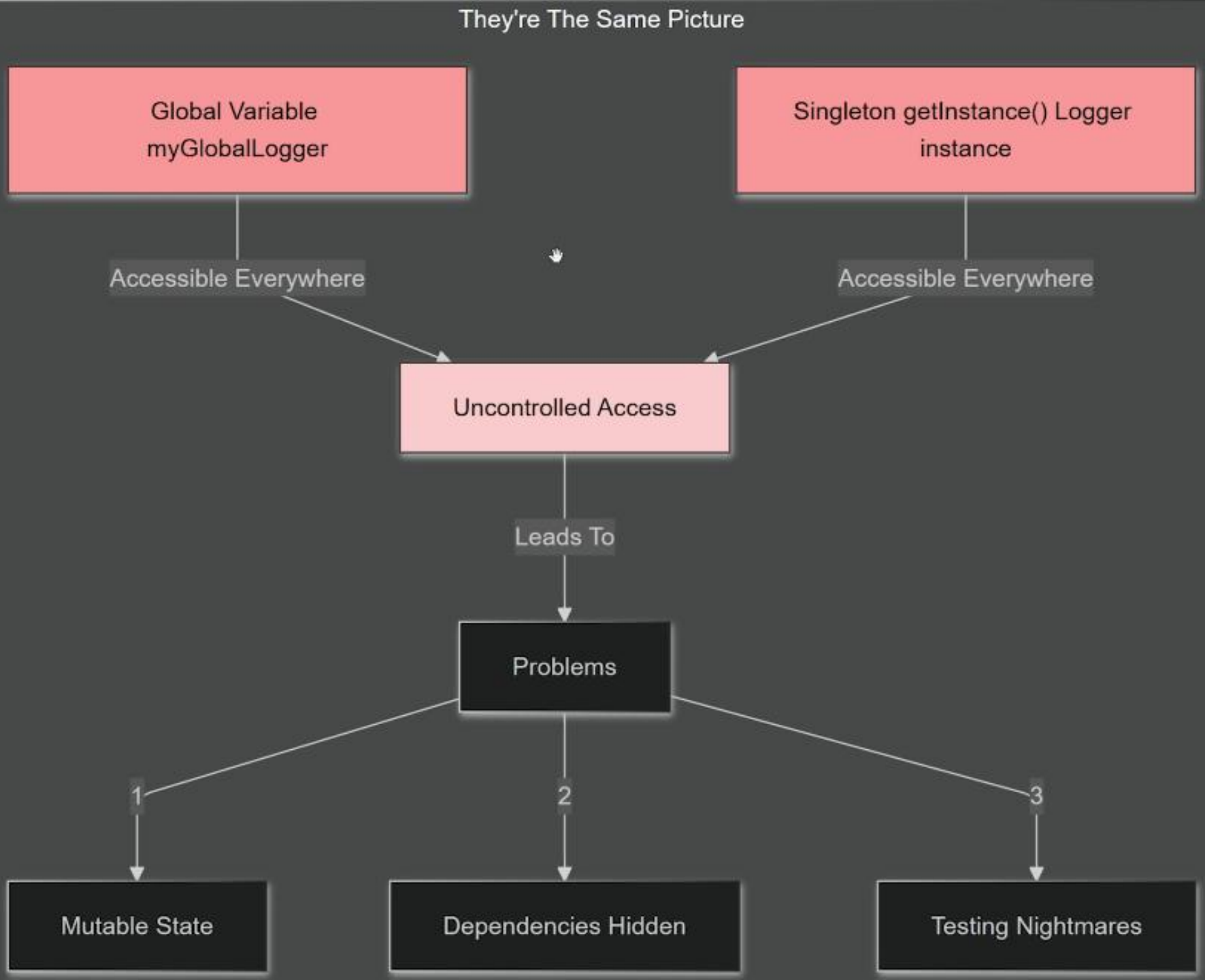
- Database Connection Pool
- Logger etc.

Singleton


Advantages
Guaranteed Single Instance
Global Access Point
Prevents multiple copies of resources/state
Easy to use anywhere in application
Controlled Resource Usage

Singleton


Challenges
Testing Difficulties
Threading Issues
Can't easily mock for unit tests
Race conditions during instance creation
Complex Test Setup
Need Thread-Safe Implementation


They're The Same Picture
Global Variable
myGlobalLogger
Singleton getInstance() Logger
instance
Accessible Everywhere
Accessible Everywhere
Uncontrolled Access
Leads To
Problems
1
2
3
Mutable State
Dependencies Hidden
Testing Nightmares

Singleton

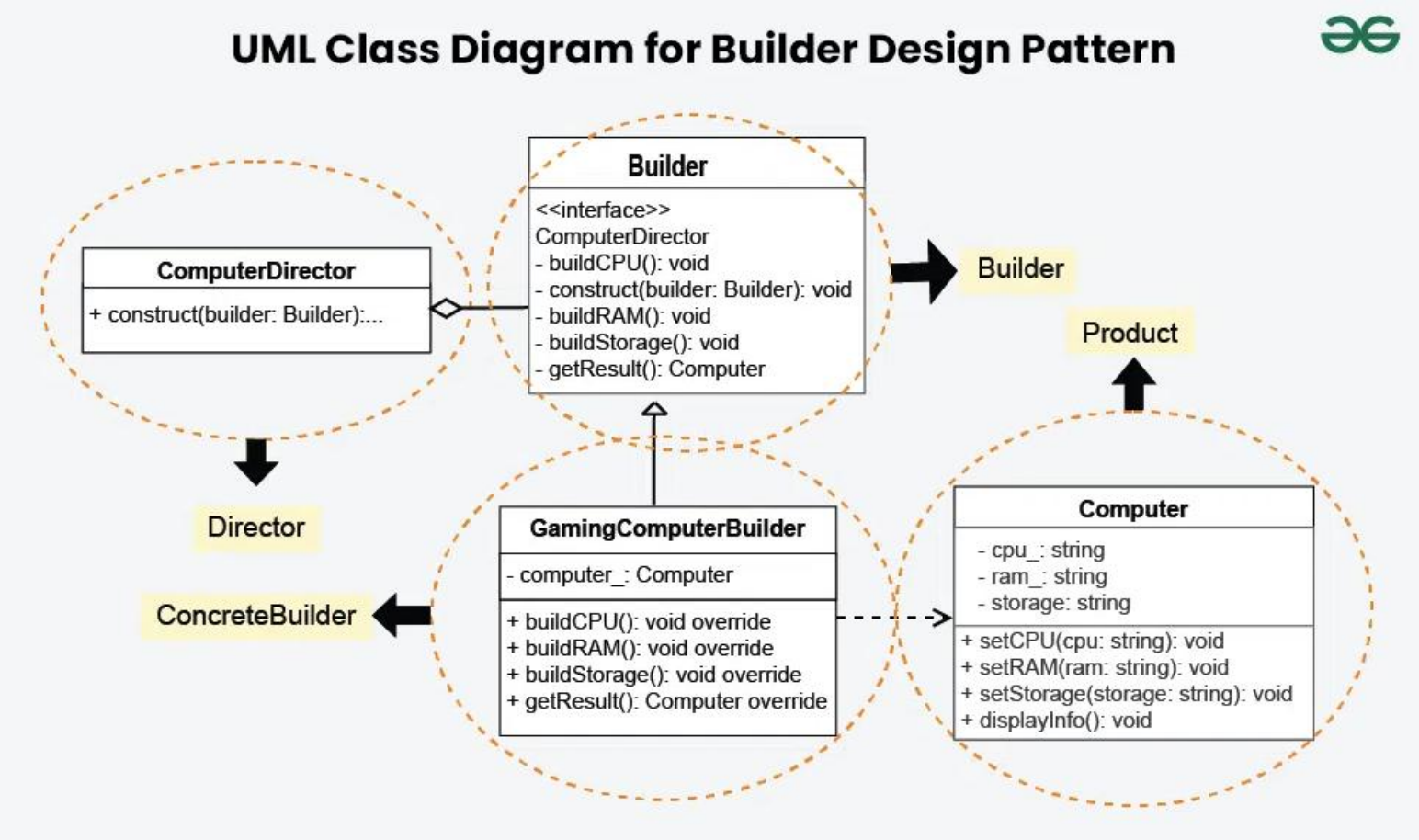
- Glorified Global Variable

Builder Pattern

- Builder pattern builds a complex object using simple objects and using a step by step approach. This type of design pattern comes under creational pattern as this pattern provides one of the best ways to create an object.

- A Builder class builds the final object step by step. This builder is independent of other objects.

Builder Pattern



Resources GFG

- https://www.geeksforgeeks.org/system-design/builder-design-pattern/
- https://refactoring.guru/design-patterns/builder
- https://www.tutorialspoint.com/design_pattern/builder_pattern.htm

Factory Pattern

- Factory pattern is one of the most used design patterns in Java. This type of design pattern comes under creational pattern as this pattern provides one of the best ways to create an object.
- In Factory pattern, we create object without exposing the creation logic to the client and refer to newly created object using a common interface.

Without Factory pattern

```php
<?php

// ------- Concrete authenticators -------
class EmailAuthenticator {
    public function authenticate($email, $password) {
        echo "Authenticating with Email + Password...\n";
        return true;
    }
}

class GoogleAuthenticator {
    public function authenticate($token) {
        echo "Authenticating with Google OAuth Token...\n";
        return true;
    }
}

class FacebookAuthenticator {
    public function authenticate($token) {
        echo "Authenticating with Facebook OAuth
Token...\n";
        return true;
    }
}
```

```php
// ------- Client Code (NO FACTORY) -------
class LoginControllerWithoutFactory {

    public function login($type, $data) {

        // Client manually decides which authenticator to
create
        if ($type === "email") {
            $auth = new EmailAuthenticator();
            return $auth->authenticate($data['email'],
$data['password']);
        }

        if ($type === "google") {
            $auth = new GoogleAuthenticator();
            return $auth->authenticate($data['token']);
        }

        if ($type === "facebook") {
            $auth = new FacebookAuthenticator();
            return $auth->authenticate($data['token']);
        }

        throw new Exception("Unknown login type.");
    }
}

// ----- Usage -----
$login = new LoginControllerWithoutFactory();
$login->login("email", [
    "email" => "test@example.com",
    "password" => "12345"
]);

?>
```

With Factory

Step 1 — Common Authenticator Interface

```
interface Authenticator {
    public function authenticate($data);
}
```

Step 2 — Concrete authenticators

```
class EmailAuthenticator implements Authenticator {
    public function authenticate($data) {
        echo "Authenticating with Email + Password...\n";
        return true;
    }
}

class GoogleAuthenticator implements Authenticator {
    public function authenticate($data) {
        echo "Authenticating with Google OAuth
Token...\n";
        return true;
    }
}

class FacebookAuthenticator implements Authenticator {
    public function authenticate($data) {
        echo "Authenticating with Facebook OAuth
Token...\n";
        return true;
    }
}
```

ULAB
UNIVERSITY OF LIBERAL ARTS BANGLADESH

With Factory

Step 3 — Creator (Factory Method)
Each factory knows how to create one authenticator.

```
abstract class AuthFactory {
    // FACTORY METHOD
    abstract public function
createAuthenticator(): Authenticator;

    // High-level login logic that uses the
product created by the factory method
    public function login($data) {
        $auth = $this-
>createAuthenticator();
        return $auth->authenticate($data);
    }
}
```

Step 4 — Concrete Factories

```
class EmailAuthFactory extends AuthFactory {
    public function createAuthenticator():
Authenticator {
        return new EmailAuthenticator();
    }
}

class GoogleAuthFactory extends AuthFactory {
    public function createAuthenticator():
Authenticator {
        return new GoogleAuthenticator();
    }
}

class FacebookAuthFactory extends AuthFactory
{
    public function createAuthenticator():
Authenticator {
        return new FacebookAuthenticator();
    }
}
```

ULAB
UNIVERSITY OF LIBERAL ARTS BANGLADESH

With Factory

Step 5 — Client Code (CLEAN)

```
// You choose the factory based on a
request, setting, or route:
$factory = new EmailAuthFactory();
// $factory = new GoogleAuthFactory();
// $factory = new FacebookAuthFactory();

$factory->login([
    "email" => "test@example.com",
    "password" => "12345"
]);
```

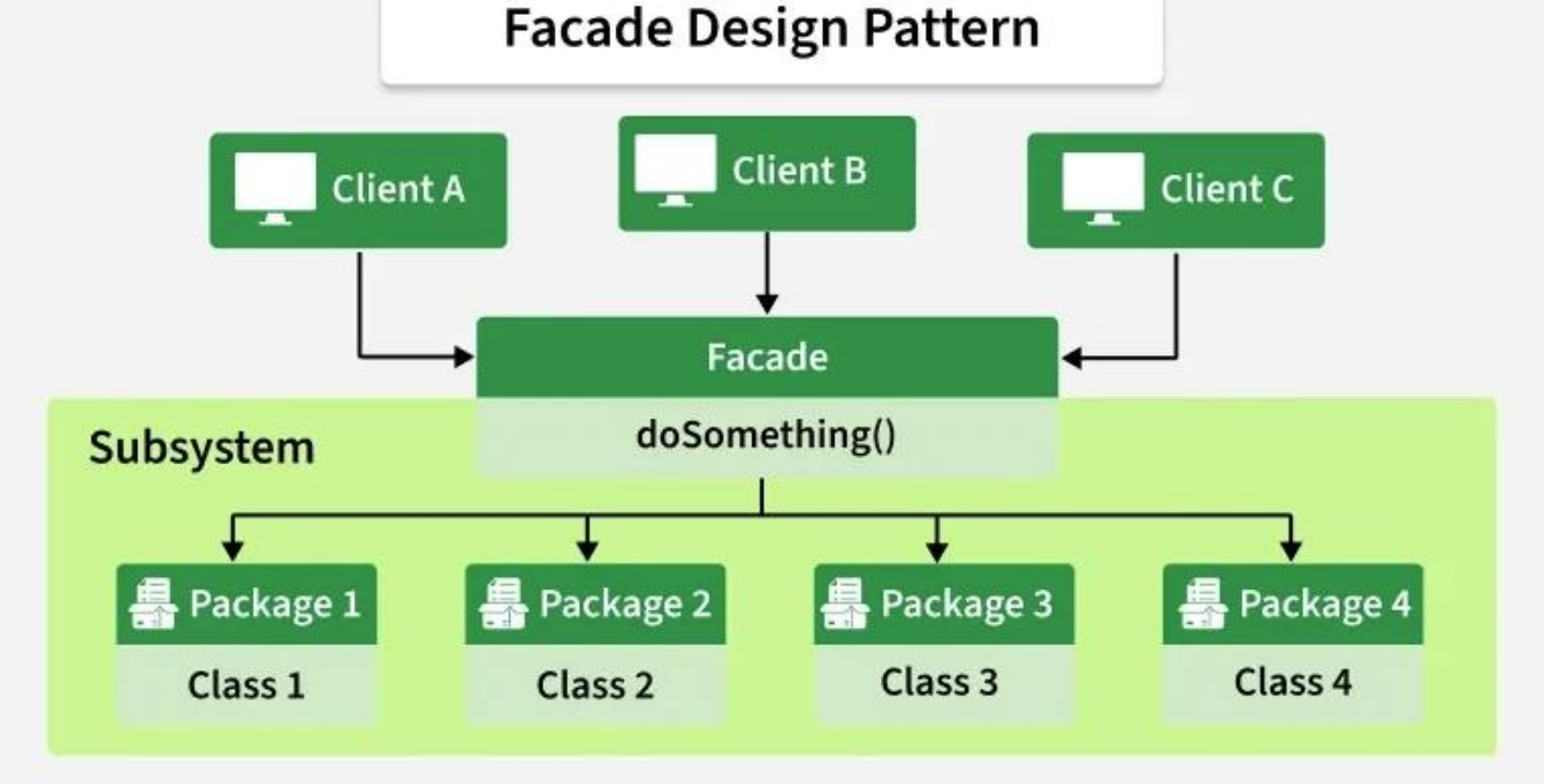
Reading

- https://refactoring.guru/design-patterns/factory-method
- https://www.tutorialspoint.com/design_pattern/factory_pattern.htm

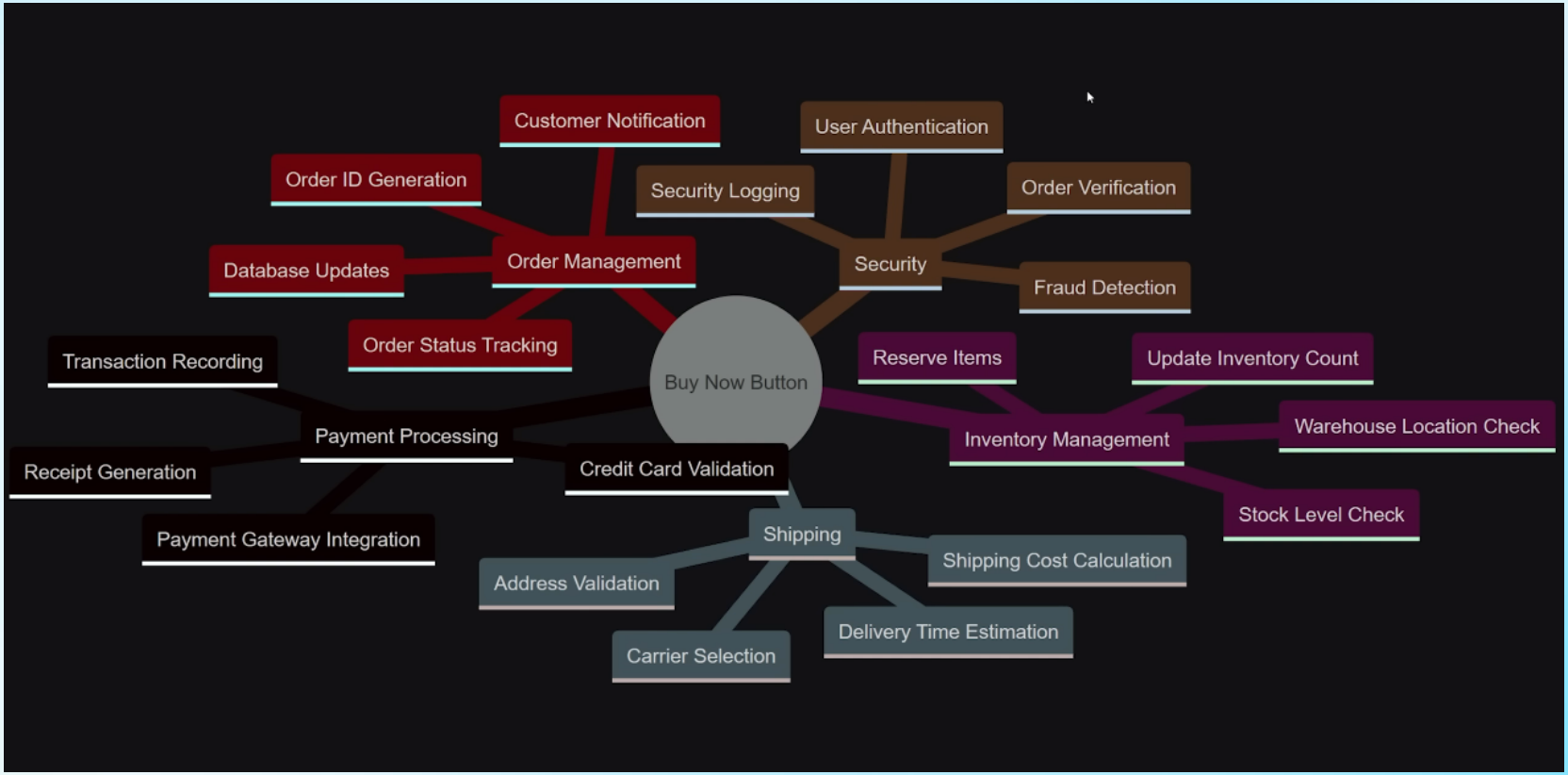
Structural Pattern

- 1. Façade
- 2. Adapter

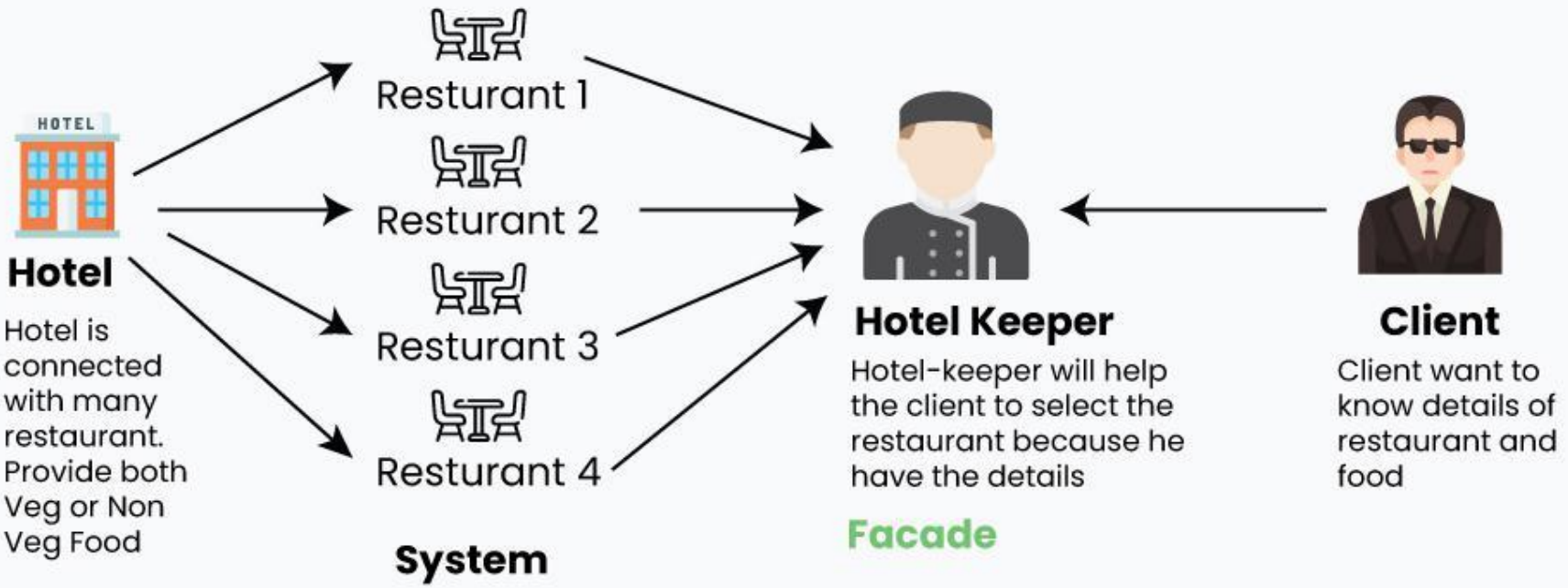
Façade Design pattern



Facade


Customer Notification
User Authentication
Order ID Generation
Security Logging
Order Verification
Database Updates
Order Management
Security
Fraud Detection
Order Status Tracking
Transaction Recording
Buy Now Button
Reserve Items
Update Inventory Count
Payment Processing
Inventory Management
Warehouse Location Check
Receipt Generation
Credit Card Validation
Stock Level Check
Payment Gateway Integration
Shipping
Shipping Cost Calculation
Address Validation
Delivery Time Estimation
Carrier Selection

Problem Statement of Facade Method Design Pattern



Without Facade

```php
<?php

// ---- Subsystems ----

class PaymentProcessor {
    public function charge($amount) {
        echo "Charging \${$amount}...\n";
        return true;
    }
}

class FraudDetector {
    public function check($userId, $amount) {
        echo "Running fraud check for User
{$userId}...\n";
        return true;
    }
}

class InventorySystem {
    public function reserveItem($productId) {
        echo "Reserving product {$productId}...\n";
        return true;
    }
}

class ShippingCalculator {
    public function calculate($address) {
        echo "Calculating shipping for {$address}...\n";
        return 5.99;
    }
}
```

```php
// ---- Client Code (NO FACADE) ----

$userId    = 1001;
$productId  = 2002;
$amount    = 49.99;
$address   = "Dhaka, Bangladesh";

$payment      = new PaymentProcessor();
$fraud        = new FraudDetector();
$inventory    = new InventorySystem();
$shipping     = new ShippingCalculator();

// Client manually coordinates everything
if($fraud->check($userId, $amount)) {
    if($inventory->reserveItem($productId)) {
        $shippingCost = $shipping->calculate($address);
        $total      = $amount + $shippingCost;

        if($payment->charge($total)) {
            echo "Purchase Completed. Total: \${$total}\n";
        }
    }
}

?>
```

With Facade

```
class BuyNowFacade {

    private $payment;
    private $fraud;
    private $inventory;
    private $shipping;

    public function __construct() {
        $this->payment   = new PaymentProcessor();
        $this->fraud     = new FraudDetector();
        $this->inventory = new InventorySystem();
        $this->shipping  = new ShippingCalculator();
    }

    public function buyNow($userId, $productId, $address,
$price) {
        echo "=== BUY NOW PROCESS STARTED ===\n";

        if(!$this->fraud->check($userId, $price)) return false;
        if(!$this->inventory->reserveItem($productId)) return false;

        $shippingCost = $this->shipping->calculate($address);
        $total        = $price + $shippingCost;

        if(!$this->payment->charge($total)) return false;

        echo "Purchase Completed Successfully! Total:
\${$total}\n";

        return true;
    }
}
```

```
// ---- Client Code (WITH FACADE) ----

$shop = new BuyNowFacade();
$shop->buyNow(
    userId: 1001,
    productId: 2002,
    address: "Dhaka, Bangladesh",
    price: 49.99
);

?>
```

Reading

- https://springframework.guru/gang-of-four-design-patterns/facade-pattern/

Adapter Design Pattern

Without Adapter (Weather API)

```php
<?php

// ----- Third-party Weather API (cannot be modified) -----
class CelsiusWeatherAPI {
    public function getTemperatureCelsius($city) {
        echo "Fetching temperature for {$city} from API...\n";
        return 30; // API always returns Celsius
    }
}


// ----- Usage -----
$client = new WeatherClientWithoutAdapter();
echo "Temperature in Fahrenheit: " .
    $client->getTemperatureFahrenheit("Dhaka") . "°F\n";

?>
```

```php
// ----- Client Code (NO ADAPTER) -----
class WeatherClientWithoutAdapter {

    private $api;

    public function __construct() {
        $this->api = new CelsiusWeatherAPI();
    }

    public function getTemperatureFahrenheit($city) {
        $tempC = $this->api->getTemperatureCelsius($city);

        // Client must convert manually every time
        $fahren = ($tempC * 9/5) + 32;

        return $fahren;
    }
}
```

With Adapter

STEP 1: Define Target Interface (What the Client Wants)

```
interface WeatherService {
   public function getTemperatureFahrenheit($city);
}
```

Step 2 — Existing API (Cannot modify)

```
class CelsiusWeatherAPI {
   public function getTemperatureCelsius($city) {
      echo "Fetching temperature for {$city} from API...\n";
      return 30;
   }
}
```

Step 3 — ADAPTER

```
class CelsiusToFahrenheitAdapter
implements WeatherService {

   private $api;

   public function
__construct(CelsiusWeatherAPI $api) {
      $this->api = $api;
   }

   public function
getTemperatureFahrenheit($city) {

      $tempC = $this->api-
>getTemperatureCelsius($city);

      // Convert behind the scenes
      $tempF = ($tempC * 9/5) + 32;

      return $tempF;
   }
}
```

Step 4 — Client now uses Fahrenheit without caring about Celsius

```
$adapter = new CelsiusToFahrenheitAdapter(new
CelsiusWeatherAPI());

echo "Temperature in Fahrenheit (via Adapter): " .
   $adapter->getTemperatureFahrenheit("Dhaka") .
"°F\n";
```

ULAB
UNIVERSITY OF LIBERAL ARTS BANGLADESH

Structural Design Pattern

- How Software Components interact with each other
- How they are structured matters

Creational Design Pattern

- How Software Components are created when being used
- How they are created matters

Behavioral Pattern

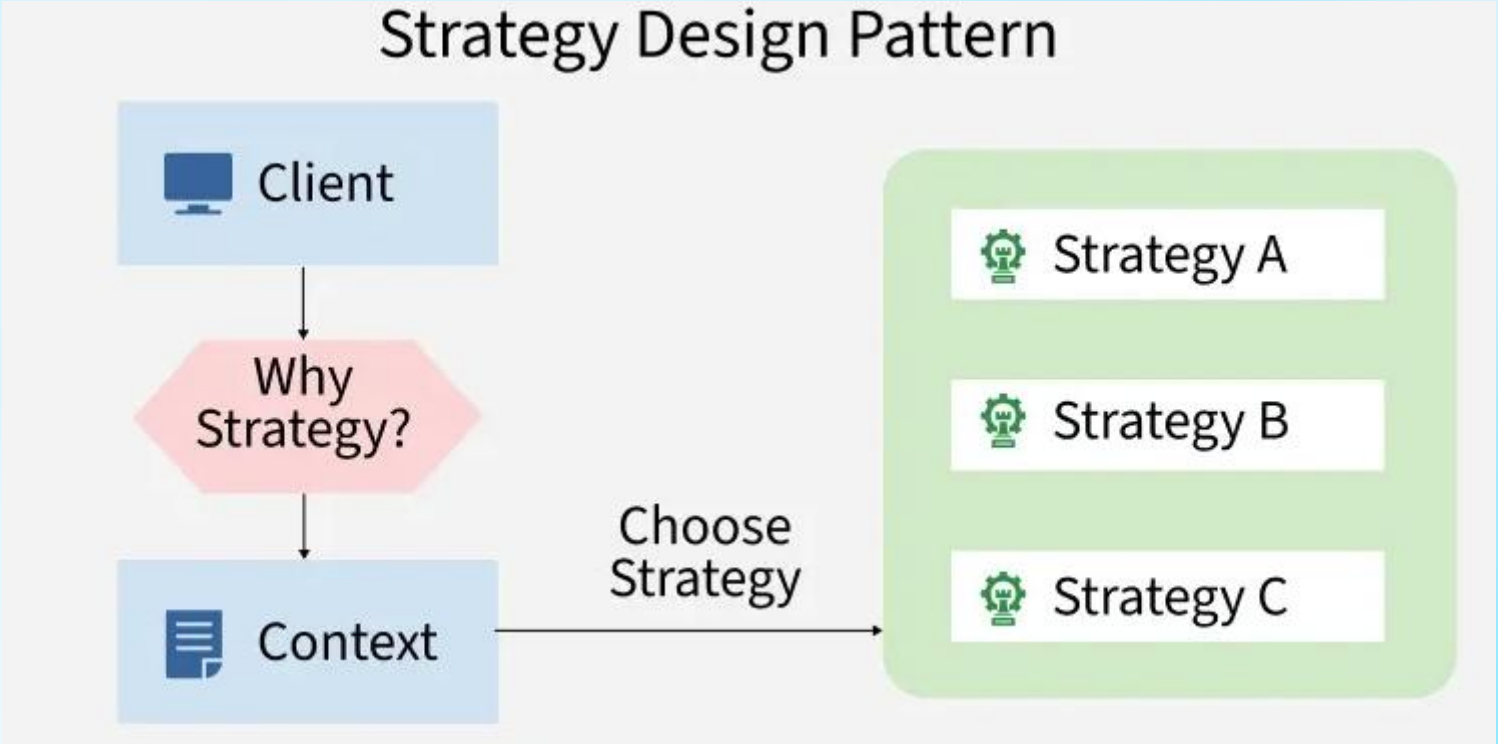
- Strategy Pattern
- Observer Pattern

What is an interface ?

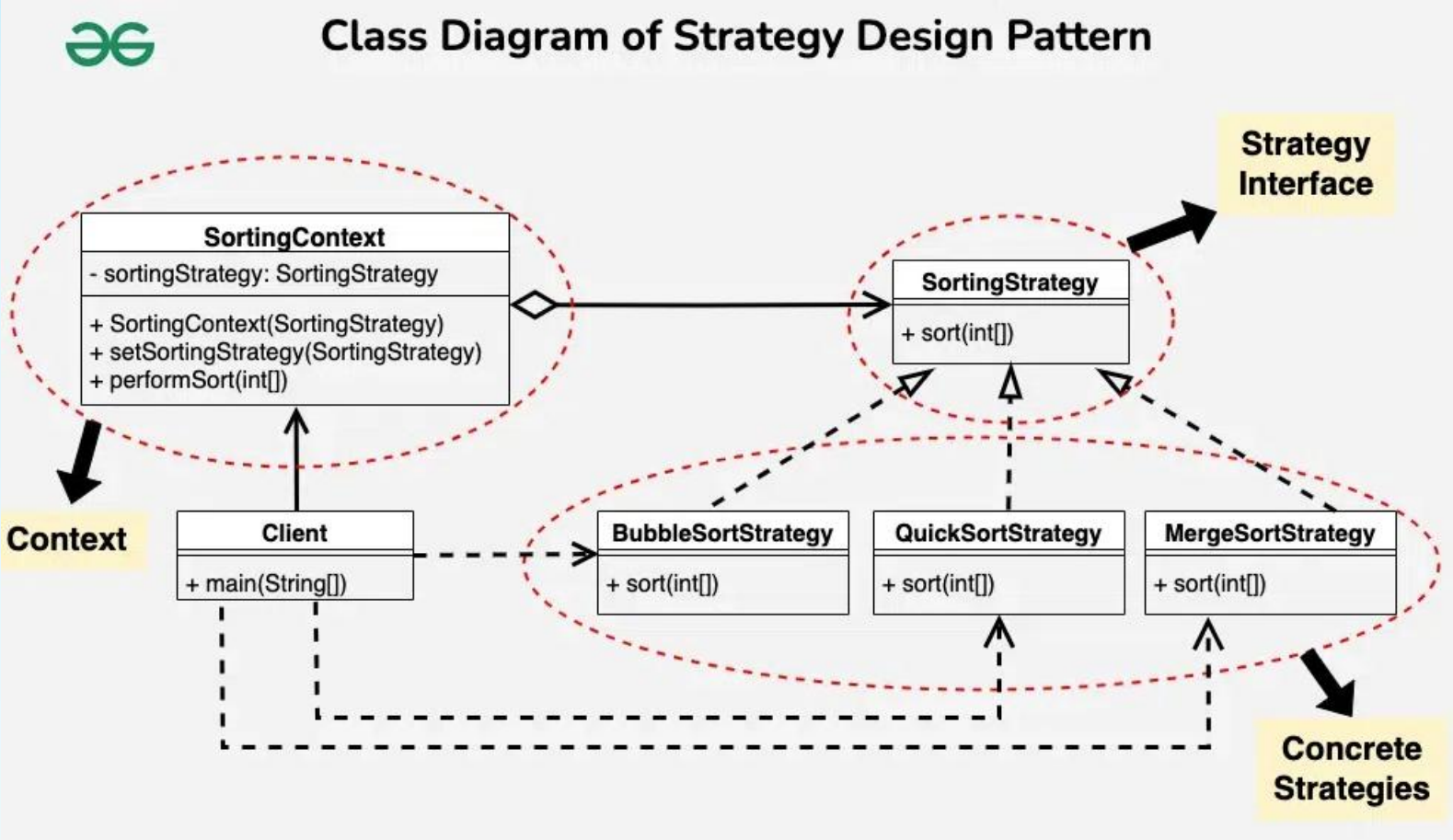
- https://stackoverflow.com/questions/2866987/what-is-the-definition-of-interface-in-object-oriented-programming

- https://en.wikipedia.org/wiki/Interface_(object-oriented_programming)

Strategy Pattern



Strategy Pattern



Code Example

1. Strategy Interface

java

Copy code

```java
// Strategy
public interface PaymentStrategy {
    void pay(int amount);
}
```

2. Concrete Strategies

Credit Card Strategy

java

Copy code

```java
public class CreditCardPayment implements PaymentStrategy {
    private String cardNumber;

    public CreditCardPayment(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    @Override
    public void pay(int amount) {
        System.out.println("Paid " + amount + " using Credit Card: " + cardNu
    }
}
```

Code Example

Bkash Payment

java

Copy code

```java
public class BkashPayment implements PaymentStrategy {
    private String phone;

    public BkashPayment(String phone) {
        this.phone = phone;
    }

    @Override
    public void pay(int amount) {
        System.out.println("Paid " + amount + " using Bkash: " + phone);
    }
}
```

Cash on Delivery

java

Copy code

```java
public class CashOnDelivery implements PaymentStrategy {
    @Override
    public void pay(int amount) {
        System.out.println("Paid " + amount + " using Cash on Delivery.");
    }
}
```

ULAB
UNIVERSITY OF LIBERAL ARTS BANGLADESH

Code Example

3. Context Class

java

Copy code

```java
// Context
public class PaymentContext {
    private PaymentStrategy strategy;

    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void checkout(int amount) {
        strategy.pay(amount);
    }
}
```

Code Example

4. Using the Strategy Pattern

java

Copy code

```java
public class Main {
    public static void main(String[] args) {
        PaymentContext context = new PaymentContext();

        // Use Credit Card
        context.setPaymentStrategy(new CreditCardPayment("1234-5678"));
        context.checkout(500);

        // Switch to Bkash
        context.setPaymentStrategy(new BkashPayment("017XXXXXXXX"));
        context.checkout(500);

        // Switch to Cash on Delivery
        context.setPaymentStrategy(new CashOnDelivery());
        context.checkout(500);
    }
}
```

Not using Strategy Pattern

```php
<?php

class PaymentProcessor {
    private string $paymentType;
    private ?string $cardNumber = null;
    private ?string $phone = null;

    public function __construct(string $paymentType, ?string
$cardNumber = null, ?string $phone = null) {
        $this->paymentType = $paymentType;
        $this->cardNumber = $cardNumber;
        $this->phone = $phone;
    }

    public function pay(int $amount): void {
        if ($this->paymentType === "credit_card") {
            if ($this->cardNumber === null) {
                echo "Error: No card number provided.\n";
                return;
            }
            echo "Paid $amount using Credit Card: {$this->cardNumber}\n";
        }
        elseif ($this->paymentType === "bkash") {
            if ($this->phone === null) {
                echo "Error: No Bkash number provided.\n";
                return;
            }
            echo "Paid $amount using Bkash: {$this->phone}\n";
        }
        elseif ($this->paymentType === "cod") {
            echo "Paid $amount using Cash on Delivery.\n";
        }
        else {
            echo "Error: Unknown payment method.\n";
        }
    }
}
```

```php
// ========================
// Demo
// ========================
$processor1 = new
PaymentProcessor("credit_card",
cardNumber: "1234-5678");
$processor1->pay(500);

$processor2 = new
PaymentProcessor("bkash",
phone: "017XXXXXXXX");
$processor2->pay(500);

$processor3 = new
PaymentProcessor("cod");
$processor3->pay(500);

?>
```

ULAB
UNIVERSITY OF LIBERAL ARTS BANGLADESH

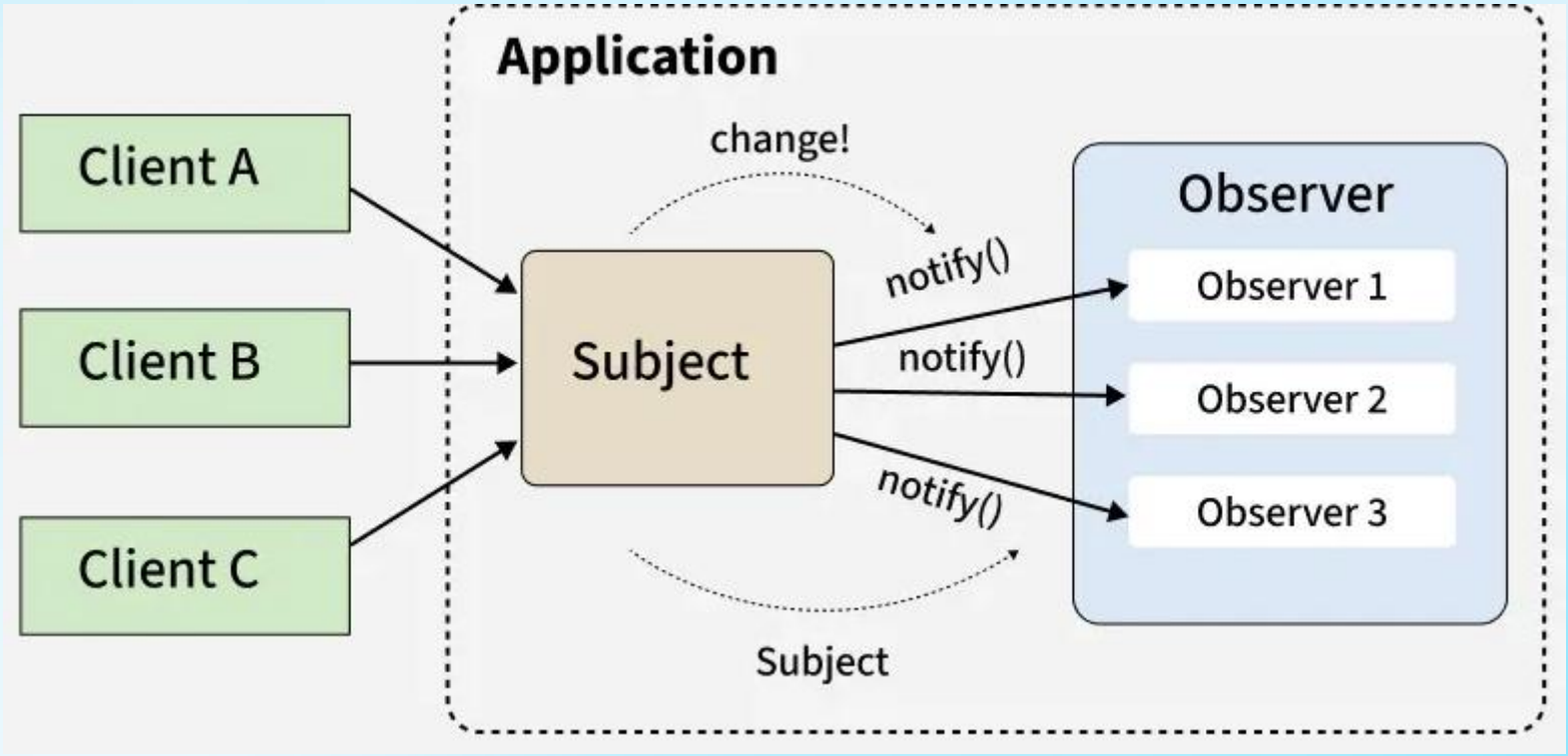
```php
<?php

// ============================
// Strategy Interface
// ============================
interface PaymentStrategy {
    public function pay(int $amount): void;
}

// ============================
// Concrete Strategies
// ============================
class CreditCardPayment implements PaymentStrategy {
    private string $cardNumber;

    public function __construct(string $cardNumber) {
        $this->cardNumber = $cardNumber;
    }

    public function pay(int $amount): void {
        echo "Paid $amount using Credit Card: {$this->cardNumber}\n";
    }
}

class BkashPayment implements PaymentStrategy {
    private string $phone;

    public function __construct(string $phone) {
        $this->phone = $phone;
    }

    public function pay(int $amount): void {
        echo "Paid $amount using Bkash: {$this->phone}\n";
    }
}

class CashOnDelivery implements PaymentStrategy {
    public function pay(int $amount): void {
        echo "Paid $amount using Cash on Delivery.\n";
    }
}
```

```php
// ============================
// Context Class
// ============================
class PaymentContext {
    private PaymentStrategy $strategy;

    public function setPaymentStrategy(PaymentStrategy $strategy): void {
        $this->strategy = $strategy;
    }

    public function checkout(int $amount): void {
        if (!isset($this->strategy)) {
            echo "Error: No payment strategy selected.\n";
            return;
        }
        $this->strategy->pay($amount);
    }
}

// ============================
// Main Script (Demo)
// ============================
$context = new PaymentContext();

// Pay using Credit Card
$context->setPaymentStrategy(new CreditCardPayment("1234-5678"));
$context->checkout(500);

// Switch to Bkash
$context->setPaymentStrategy(new BkashPayment("017XXXXXXXX"));
$context->checkout(500);

// Switch to Cash on Delivery
$context->setPaymentStrategy(new CashOnDelivery());
$context->checkout(500);

?>
```

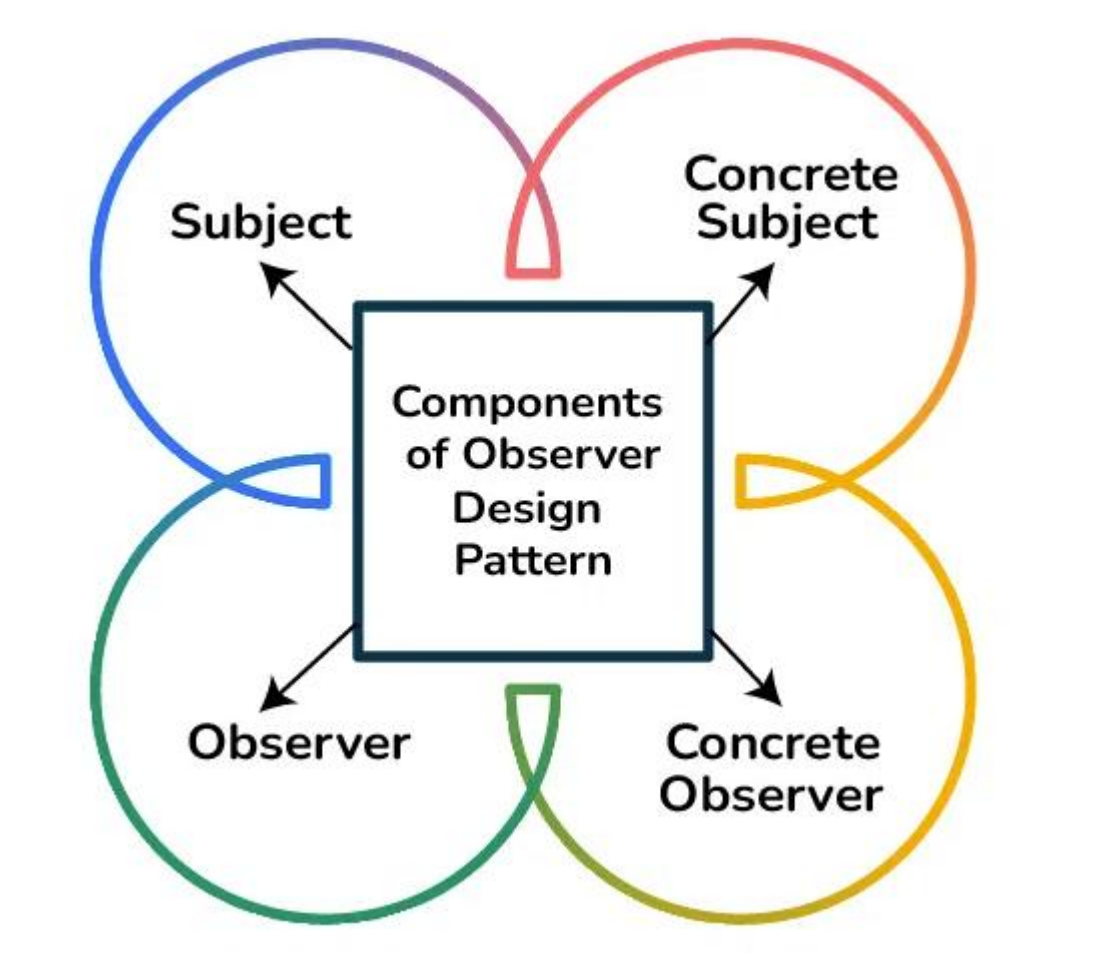
Resource

- https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/

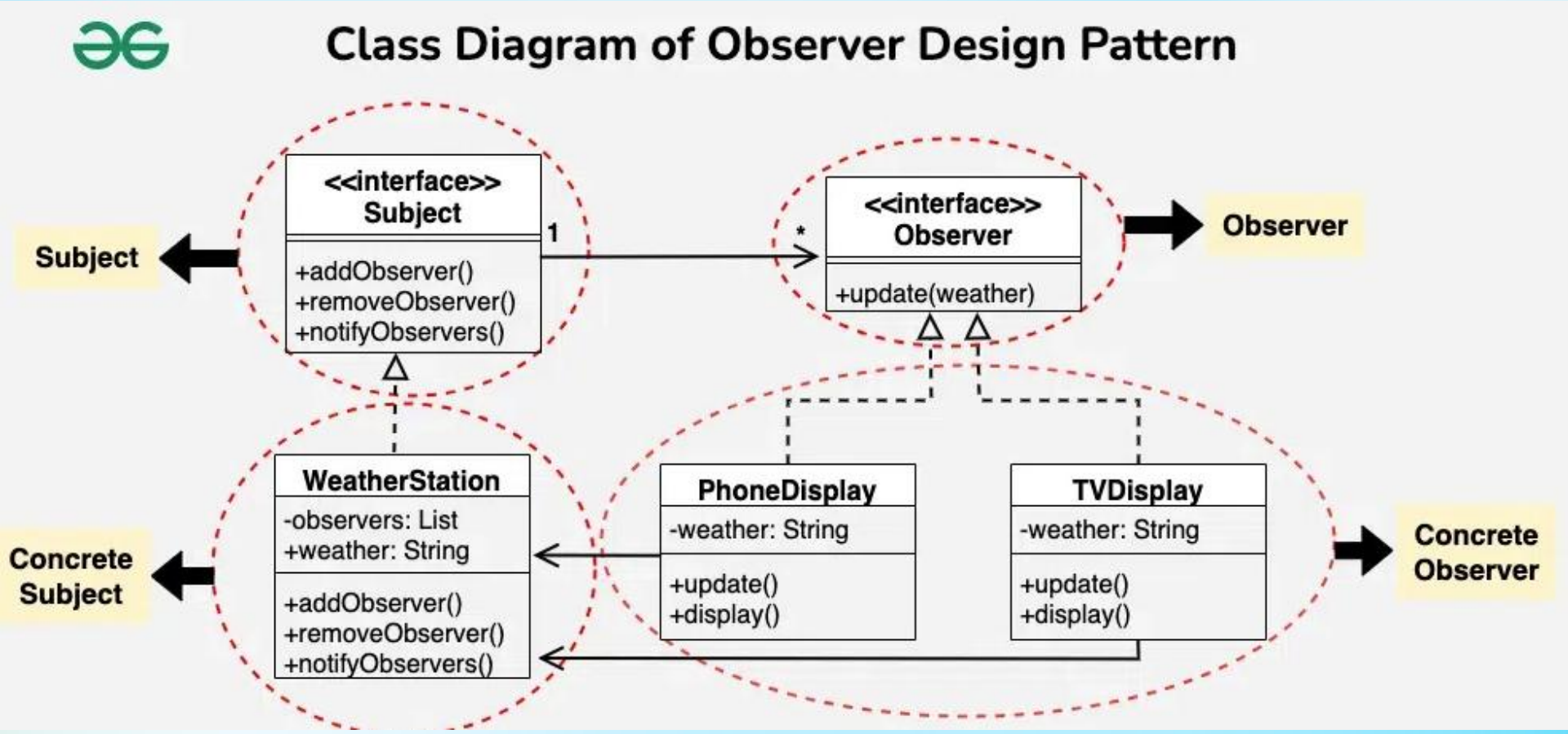
Observer Pattern



Observer Pattern



Observer Pattern



Without Observer

```php
<?php

class YouTubeChannel {

    public function uploadVideo(string $title): void {
        echo "Channel uploaded a new video: $title\n";

        // Hard-coded subscribers
        $this->notifyAlice($title);
        $this->notifyBob($title);
    }

    private function notifyAlice(string $title): void {
        echo "Notifying Alice: New video - $title\n";
    }

    private function notifyBob(string $title): void {
        echo "Notifying Bob: New video - $title\n";
    }
}

// Demo
$channel = new YouTubeChannel();
$channel->uploadVideo("Observer Pattern Explained");
```

With Observer

```php
<?php

//
// ============================
// Observer Interface
//
// ============================
interface Subscriber {
    public function update(string
$videoTitle): void;
}

//
// ============================
// Concrete Subscribers
//
// ============================
class User implements Subscriber {
    private string $name;

    public function __construct(string
$name) {
        $this->name = $name;
    }

    public function update(string
$videoTitle): void {
        echo "{$this->name} received
notification: New video uploaded -
$videoTitle\n";
    }
}
```

```php
// ============================
// Subject (Observable)
// ============================
class YouTubeChannel {
    private array $subscribers = [];

    public function subscribe(Subscriber
$subscriber): void {
        $this->subscribers[] = $subscriber;
    }

    public function
unsubscribe(Subscriber $subscriber):
void {
        $this->subscribers = array_filter(
            $this->subscribers,
            fn($s) => $s !== $subscriber
        );
    }

    public function uploadVideo(string
$title): void {
        echo "Channel uploaded a new
video: $title\n";
        $this->notifySubscribers($title);
    }

    private function
notifySubscribers(string $title): void {
        foreach ($this->subscribers as
$subscriber) {
            $subscriber->update($title);
        }
    }
}
```

```php
//
// ============================
// Demo
//
// ============================
$channel = new YouTubeChannel();

$alice = new User("Alice");
$bob   = new User("Bob");
$charlie = new User("Charlie");

$channel->subscribe($alice);
$channel->subscribe($bob);
$channel->subscribe($charlie);

$channel->uploadVideo("Observer
Pattern Explained");

// Unsubscribe Bob
$channel->unsubscribe($bob);

$channel->uploadVideo("Strategy
Pattern Tutorial");
```

ULAB
UNIVERSITY OF LIBERAL ARTS BANGLADESH

Resources

- https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/

Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh

Department of Computer Science & Engineering